Analýza a zložitosť algoritmov – 2025/2026

Praktické zadanie

1. Greedy algoritmus

Greedy algoritmus pre plánovanie jobov s deadlinami funguje tak, že najskôr zoradí joby podľa profitu v zostupnom poradí. Na to sa použije pole **jobOrder**, kde sú uložené indexy jobov 1..n a štandardná funkcia **sort** s porovnávaním podľa poľa profit. Výsledkom je, že **jobOrder[1]** je job s najväčším profitom, **jobOrder[2]** je druhý najväčší atď.

Postupnosť plánovaných jobov sa ukladá do štruktúry **sequence_of_integer** ako zoznam pozícií v zoradenom poli. Pri prechode zoradeného zoznamu jobov sa každý nový job skúsi vložiť do aktuálneho rozvrhu tak, aby sa zachovalo neklesajúce usporiadanie podľa deadlinov. To zabezpečuje metóda **insertInOrder**, ktorá najprv nájde pozíciu, kam nový job patrí a následne posunie všetky neskoršie prvky doprava, aby vzniklo miesto na vloženie.

Po každom vložení sa zavolá funkcia **isFeasible**, ktorá simuluje rozvrhnutie jobov tak, že job na pozícii i je naplánovaný v čase i. Pre každý job sa overí podmienka, že jeho deadline je väčší alebo rovný číslu slotu, na ktorý bol priradený. Ak sa zistí, že nový job porušuje deadline (teda žiaden uskutočniteľný rozvrh pre aktuálnu množinu jobov neexistuje), postupnosť J sa vráti do pôvodného stavu a nový job sa ignoruje. Na konci algoritmu je v J uložená množina jobov, ktorá prináša maximálny profit pri dodržaní deadlinov a tá sa v **maine** premietne späť na pôvodné indexy jobov a vypíše sa výsledná sekvencia aj súčet profitu.

Zložitosti (T a O – uvádzam ju vždy pod časovou zložitosťou):

*sequence_of_integer& **operator**=(const sequence_of_integer& other)*

- vo funkcii sa nachádza 1 for-cyklus, od 0 do size

T(d) = $\sum_{i=0}^{d-1} 1$ = d

O(d)

*void **insertInOrder**(int job, const int deadline[])*

- vo funkcii sú 2 for-cykly

1. cyklus: **T(d)** = $\sum_{i=0}^{d-1} 1$
2. cyklus: **T(d)** = $\sum_{i=X}^{d} 1$

- prvý cyklus skončí pri indexe **X** kvôli podmienke vo vnútri cyklu, druhý cyklus začína od konca a zastaví sa na indexe **X**, preto celková suma tejto funkcie je T(d) = $\sum_{i=0}^{d} 1$= d+1

O(d)

*bool **isFeasible**(const sequence_of_integer& J, const int deadline[])*

- funkcia obsahuje len 1 for-cyklus od 0 po veľkosť J

T(d) = $\sum_{i=0}^{d-1} 1$ = d

O(d)

*void **schedule**(int n, const int deadline[], sequence_of_integer& J)*

- funkcia schedule je hlavnou časťou celého programu, obsahuje 1 for-cyklus v ktorom sa volajú vyššie uvedené funkcie

T(n) = $\sum_{i=2}^{n}(4d+1) = (n-1)(4d+1) = 4nd - 4d + n - 1$

O(n^2)

- d = počet deadlinov
- n = počet jobov

Vo funkcii ***int main()*** mám ešte 3 cykly.

1. cyklus slúži na inicializáciu poľa jobOrder, časová zložitosť tohoto cyklu je **T(n)** = $\sum_{i=1}^{n} 1 \ = n$

O(n)

2. cyklus slúži na mapovanie zotriedených Jobov podľa profitu a pôvodného zoradenia Jobov, časová zložitosť tohoto cyklu je taktiež **T(n)** = $\sum_{i=1}^{n} 1 \ = n$

O(n)

3. cyklus slúži na výpis najlepšieho riešenia **T(d)** = $\sum_{i=0}^{d} 1 \ = d + 1$

O(d)

Vo funkcii ***main*** sa volá aj funkcia ***schedule***, ktorej časová zložitosť je 4nd $- 4d + n - 1$.

Celková časová zložitosť: **T(n)** = 4nd $- 3d + 3n$

Z tohoto už vidím asymptotickú zložitosť (hornú hranicu):

Pre krok n:

4nd $-3d+3n \in O(n^2)$, pretože platí

$$4\text{nd} - 3d + 3n \leq 4n^2 + 3n \leq 7n^2 \in O(n^2)\ s\ n_0 = 1\ a\ c = 7, d \geq 1, d \leq n$$

Pre krok n + 1:

$$4(n+1)d - 3d + 3(n+1) \leq 7(n+1)^2$$
$$4nd + 4d - 3d + 3n + 3 \leq 7n^2 + 14n + 7$$

$$4nd + d + 3n + 3 \leq 7n^2 + 14n + 7$$

$$4nd + d \leq 7n^2 + 11n + 4$$

Keďže d ≤ n ,tak asymptotická horná hranica zložitosti celého algoritmu je $O(n^2)$.

Tento program je možné zlepšiť použitím vhodnejšej dátovej štruktúry, viď. časť 2 – disjoint sets.

2. Disjoint sets

V tejto implementácii sa na plánovanie jobov s deadlinami používa dátová štruktúra disjunktných množín. Každý časový slot od 1 do d je na začiatku samostatný set reprezentovaný uzlom **nodetype**, ktorý obsahuje index svojho rodiča, hĺbku stromu a informáciu smallest, ktorá uchováva index najbližšieho voľného slotu v danom strome.

Na začiatku sa pre všetky sloty zavolá **make_set**, čím sa inicializujú stromy tak, že parent[i] = i a smallest = i. Joby zoradím podľa profitu v zostupnom poradí. Pre každý job s deadlinom d sa zavolá **find_set(d)**, čím získam root množiny, kt. reprezentuje interval slotov, kde ešte môže byť tento job naplánovaný. Z množiny sa prečíta smallest, čo je index najbližšieho voľného slotu $\leq$ d. Ak je smallest kladný, job sa umiestni do **resultArr[smallest]** a následne sa pomocou **merge** spoja množiny pre slot smallest a smallest - 1. Tým sa označí, že slot smallest je obsadený a pri ďalších joboch sa ako najbližší voľný slot bude ponúkať ešte menší index. Po spracovaní všetkých jobov obsahuje pole **resultArr** optimálny rozvrh podľa profitu, z ktorého sa v **maine** vypočíta celkový profit a vypíše sa výsledná sekvencia jobov.

Zložitosti (T a O – uvádzam ju vždy pod časovou zložitosťou):

*index **find_set**(index x)*

- táto funkcia využíva rekurziu, keďže sa jedná o stromovú dátovú štruktúru, takže časová aj asymptotická zložitosť (horná hranica) je logaritmická

T(d) = log d

O(log d)

*void **scheduleJobs**(Job jobs[], int n, int maxDeadline)*

- funkcia ***scheduleJobs*** využíva oproti 1. implementácii stromovú dátovú štruktúru

1. cyklus ide od 0 po maxDeadline -> d, **T(d)** = $\sum_{i=0}^{d} 1 \;=\; d+1$

O(d)

2. cyklus ide od 0 do n – 1 a v každej iterácii cyklu volám funkciu ***find_set*** najviac trikrát, **T(n)** = $\sum_{i=0}^{n-1} 3\log d \;=\; 3\sum_{i=0}^{n-1}\log d \;=\; 3\,n\,\log d$

O(n log d)

Vo funkcii ***int main()*** mám ešte 2 cykly.

1. cyklus od 0 po n-1, čiže **T(n)** = $\sum_{i=0}^{n-1} 1 = n$

A 2. cyklus na výpis najlepšieho rozvrhu, cyklus prechádza cez deadliny, **T(n)** = $\sum_{i=1}^{d} 1 = d$

Vo funkcii ***main()*** sa volá aj funkcia ***scheduleJobs()***, ktorá má zložitosť $3n \log d$, spolu teda **T(n)** = $3n \log d + n + 2d + 1$.

Chcem dokázať, že algoritmus má asymptotickú tesnú hranicu zložitosti θ(nlogm), čiže musím dokázať, že patrí do Ω(nlogm) a súčasne do O(nlogm).

Asymptotickú hornú hranicu zložitosti O dokážem obdobne ako v 1. implementácii. Nech m je *min(n, d)*.

Pre krok n:

$$3n \log m + n + 2m + 1 \leq 3n \log m + 4n \leq 7n \log m$$
$$7n \log m \in O(n \log m) \; s \; n_0 = 1, m \geq 2 \; a \; c = 7$$

Pre krok $n + 1$:

$$3(n+1) \log m + n + 2m + 2 \leq 7(n+1) \log m$$
$$3n \log m + 3 \log m + n + 2m + 2 \leq 7n \log m + 7 \log m$$

Jasne vidím, že na ľavej strane nerovnice je 3nlogm + n + 2m, čo bude vždy menšie alebo rovné 7nlogm, n je menšie ako nlogm a zároveň 2m bude nanajvýš rovné 2n, čo je menšie ako 2nlogm. Zároveň na ľavej strane je 3logm + 2 čo je menšie ako 7logm pre m ≥ 2.

Teraz dokážem asymptotickú dolnú hranicu zložitosti Ω.

Pre krok n:

$$nlog\, m \leq 3nlogm + n + 2m + 1, \; n_0 = 1, m \geq 1 \; a \; c = 1$$

Pre krok n + 1:

$$(n+1) \log m \leq 3(n+1) \log m + n + 2m + 2$$

$$n \log m + \log m \leq 3n \log m + 3 \log m + n + 2m + 2$$

Jasne vidím, že ľavá strana nerovnice bude vždy menšia ako pravá nakoľko na ľavej mám nlogm, zatiaľ čo na pravej 3nlogm a na ľavej log m, zatiaľ čo na pravej 3logm.

Týmto som dokázala, že algoritmus na scheduling s použitím disjoint setov má asymptotickú tesnú hranicu zložitosti θ(nlogm).

Môžem program zlepšiť ak použijem stromy, ktoré budú plytké, tj. listy budú napojené priamo na koreň. Tým pádom ušetrím operácie na prechod stromom a zlepším operáciu find_set. V tomto prípade bude zložitosť find_set inverzná Ackermannova funkcia, ktorá má asymptotickú hornú hranicu zložitosti **O(α(n))**.

Ukážka výstupu na vstupe zo zadania:

1. *schedulingNikolaSasinkova.cpp*

```
Scheduled Jobs: 7 1 3 2
Maximum total profit = 170
```

2. *scheduling2NikolaSasinkova.cpp*

```
Scheduled Jobs: 7 1 3 2
Maximum total profit = 170
```

3. Huffman (návrh riešenia)

a. Kompresia

1. Súbor načítam celý v binárnom režime do premennej text typu string
2. Pre všetkých 256 znakov spočítam frekvencie do freqMap
3. Na základe frekvencií vytvorím Huffmanov strom (buildTree) a z neho vygenerujem kódy codes
4. V hlavičke súboru uložím:
    - Počet bitov zakódovaného textu (textBits)
    - Počet bitov na zápis frekvenčnej tabuľky
    - Pre každý znak 0..255:
        - Počet bitov potrebných na zápis jeho frekvencie
        - Samotnú frekvenciu v binárnom tvare - zabalenú po bajtoch (s paddingom do 8 bitov)
5. Telo súboru tvorí bitový stream Huffman kódov - postupne prechádzam znaky vstupu, skladám ich kódy do buffra a každých 8 bitov zapisujem ako jeden bajt

b. Dekompresia

1. Z hlavičky súboru načítam textBits, headerBits a pre každý znak obnovím frekvenciu (z počtu bitov a binárneho zápisu frekvencie)
2. Z frekvencií znova postavím Huffmanov strom (buildTree)
3. Zvyšok súboru načítam po bajtoch, prevediem na bitový reťazec a sekvenčne ho prechádzam tak, že zároveň sa pohybujem v strome, pri dosiahnutí listu v strome zapíšem príslušný znak do výstupu
4. Výsledkom je pôvodný súbor pred kompresiou

Huffmanov strom

Huffmanov strom sa buduje na základe frekvenčnej tabuľky znakov, ktorú program uchováva v slovníkoch **freqMap** a **codes**. Najprv sa pre každý ASCII znak (0..255) spočíta frekvencia výskytu v súbore. Funkcia **buildTree()** potom prejde všetky znaky a pre tie, ktoré majú kladnú frekvenciu, vytvorí listové uzly typu **Node** a vloží ich do priority queue, usporiadanej podľa frekvencie. Pokiaľ je queue prázdna, výsledkom je prázdny strom.

Ak v priority queue existuje aspoň jeden prvok, algoritmus opakovane vyberá dva uzly s najnižšou frekvenciou, vytvorí nový vnútorný uzol, ktorého frekvencia je súčtom frekvencií týchto dvoch uzlov a nastaví ich ako ľavého a pravého syna. Nový uzol vloží späť do priority queue. Tento proces sa opakuje, kým v queue nezostane jediný uzol - ten sa stáva koreňom Huffmanovho stromu. Následne funkcia **generateCodes** rekurzívne prechádza strom - pri pohybe doľava pridá do aktuálneho kódu bit 0, pri pohybe doprava bit 1. Pri návšteve listu sa vzniknutý bitový reťazec uloží do mapy **codes[ch]**. Výsledkom je prefixový kód, ktorý jednoznačne priraďuje každému znaku reťazec bitov podľa jeho frekvencie vo vstupných dátach.

Zložitosti (T a O – uvádzam ju vždy pod časovou zložitosťou):

Node **buildTree**()*

Do stromu vkladám najviac 256 prvkov, teda **T(n)** = $\sum_{i=0}^{255} 1 = 256$.

Potom while-cyklom vyberiem 2 prvky, vytvorím im spoločný parent Node a vložím tento Node naspäť do priority queue. V každej iterácii mi z priority queue ubudne 1 prvok, čiže **T(n)** = $\sum_{i=0}^{254} 1 = 255$.

Spolu **T(n)** = 511

*void **generateCodes**(Node* node, string code)*

Vygenerovanie kódov znamená prechod cez celý strom, prechod cez všetky Nody **T(n)**=$\sum_{i=0}^{510} 1 = 511$ - počet všetkých Nodov v úplnom strome, ktorý ma 256 listov.

Asymptotická horná hranica zložitosti je v oboch prípadoch konštanta, čiže **O(1)**.

a. Kompresia

Funkcia **compress** najskôr vynuluje frekvenčnú tabuľku **freqMap** pre všetkých 256 ASCII znakov. Následne v binárnom režime načíta celý vstupný súbor do premennej text typu string a v jednom prechode spočíta, koľkokrát sa každý znak vyskytuje vo vstupnom súbore. Na základe tejto frekvenčnej tabuľky sa zavolá funkcia **buildTree()** na vytvorenie Huffmanovho stromu a funkcia **generateCodes()**, ktorá pre každý znak vypočíta jeho binárny Huffmanov kód.

Pred samotným zápisom dát do súboru sa pre každý znak spočíta, aký bude celkový počet bitov zakódovaného textu - k premennej **textBits** sa pripočíta **freq * dĺžka kódu**, kde **freq** je frekvencia znaku a **dĺžka kódu** je počet bitov v jeho Huffmanovom kóde. Premenná **textBits** sa uloží ako 64-bitové celé číslo na začiatok súboru. Následne uložím ako uint64 premennú **headerBits**, v ktorej sa nachádza veľkosť zápisu frekvenčnej tabuľky v bitoch. Potom sa pre každý ASCII znak zapíše do hlavičky počet bitov potrebných na binárny zápis jeho frekvencie (1 bajt) a samotná frekvencia zakódovaná v danej dĺžke, rozdelená na bajty a doplnená nulami do násobku 8 bitov. Tým získa dekompresor všetky informácie potrebné na zrekonštruovanie frekvenčnej tabuľky. Ak je frekvencia nejakého znaku 0, namiesto zápisu 0000 1000 ako počet potrebných bitov na zápis frekvencie a 0000 0000 ako frekvenciu, zapíšem len 0000 0000.

Po hlavičke nasleduje samotné zakódovanie textu - pre každý znak vstupu sa do dočasného reťazca buffer pridá jeho kód z mapy **codes**. Vždy, keď buffer obsahuje aspoň 8 bitov, prvých 8 bitov sa prevedie pomocou bitset<8> na jeden bajt a zapíše do súboru. Na konci, ak v bufferi ostali nejaké bity, doplnia sa nulami do plných 8 bitov, skonvertujú sa na posledný bajt a ten sa taktiež zapíše. Výsledkom je binárny súbor, ktorý na začiatku obsahuje hlavičku s počtom platných bitov textu, veľkosťou zápisu frekvenčnej tabuľky v bitoch a popisom frekvencií a za ňou samotný komprimovaný bitový stream.

Zložitosti (T a O – uvádzam ju vždy pod časovou zložitosťou):

- f = frekvencia

*int **bitsForFreq**(int f)*

Funkcia ***bitsForFreq()*** vráti počet bitov, ktoré potrebujem na zakódovanie intu f do binárnej sústavy. Vo funkcii je jeden while cyklus, ten sa vykoná log f-krát.

T(f) = $\sum_{i=1}^{\log f} 1 = \log f$

O(log f)

void ***compress****(string inputFile, string outputFile)*

Funkcia ***compress*** vykonáva spolu 5 for-cyklov a 3 while-cykly.

1. for-cyklus inicializuje frekvenčnú mapu pre všetky ASCII znaky, teda **T(n)** = $\sum_{i=0}^{255} 1 = 256$

O(1)

2. while-cyklus, načíta celý obsah súboru do stringu, čiže **T(n)**=$\sum_{i=1}^{n} 1 = n$

O(n)

3. for-cyklus prejde text a vybuduje frekvenčnú tabuľku, **T(n)**=$\sum_{i=0}^{n-1} 1 = n$

O(n)

Z tabuľky vybudujem strom, **T(n)** = 511 ∈ **O(1)**.

Taktiež sa po tomto kroku vygenerujú kódy: **T(n)** = 511 ∈ **O(1)**

4. for-cyklus spočíta počet bitov potrebných na zakódovanie, pre každý znak ASCII a jeho frekvenciu viem jeho kód, vynásobím frekvenciu znaku a dĺžku jeho kódu, sčítam cez všetky znaky, **T(n)** = $\sum_{i=0}^{255} 1 = 256$

O(1)

5. for-cyklus slúži na zápis frekvencií, **T(n)** = $\sum_{i=0}^{255} 1 = 256$

   a. V rámci tejto funkcie volám aj ***bitsForFreq()***: **T(f)** = log f
   b. For-cyklus, ktorý prechádza cez bity a vytvára zápis f v binárnej sústave **T(n)** = log f
   c. While-cyklus, ktorý zapisuje bity v binárnej sústave po 8 do súboru, **T(n)** = $\frac{\log f}{8}$ alebo sa vykoná while-cyklus, ktorý doplní binárny zápis do 8 bitov nulami, **T(n)** = 7 (najhorší prípad)

O(log f)

6. for-cyklus cez text, **T(n)** = $\sum_{i=0}^{n-1} 1 = n$

O(n)

7. while-cyklus - zapisujem bity po 8 do výstupného súboru, v najhoršom prípade to bude n zápisov do súboru, **T(n)** = $\sum_{i=0}^{n-1} 1 = n$

O(n)

8. Doplnenie bufferu do 8 bitov while-cyklom, **T(n)** = 7

O(1)

Celková časová zložitosť **T(n)** = 256 + 2n + 1022 + 256 + 256 (log f + log f / 8) + n + n + 7 = $4n + 288 \log f + 1541$

Asymptotickú hornú hranicu zložitosti O dokážem z T(n):

Pre krok n:

$$4n + 288 \log f + 1541 \leq 1833n \in O(\mathrm{n})\ s\ c = 1833\ a\ n_0 = 1, f \leq n, f \geq 0$$

Pre krok n+1:

$$4(n+1) + 288 \log f + 1541 \leq 1833(\mathrm{n}+1)$$

$$4n + 4 + 288 \log f + 1541 \leq 1833n + 1833$$

$$4n + 288 \log f + 1545 \leq 1833n + 1833\ /-4n - 1545$$

$$288 \log f \leq 1829n + 288$$

Počet vstupných znakov n bude vždy väčší ako logaritmus z frekvencie znaku.

Takže asymptotická horná hranica zložitosti celého algoritmu je $\boldsymbol{O(n)}$.

V programe môžem nahradiť udržiavanie bytu ako bit bufferu namiesto stringu a charu a tým by som mohla zlepšiť program.

a. Dekompresia

Pri dekompresii sa ako prvé z komprimovaného súboru načíta 64-bitové číslo **textBits**, ktoré udáva počet platných bitov zakódovaného textu (bez paddingových núl na konci). Potom sa prečíta veľkosť zápisu frekvenčnej tabuľky v bitoch. Následne sa vynuluje frekvenčná tabuľka **freqMap**. Hlavička obsahuje informáciu o frekvencii každého z 256 ASCII znakov: Pre každý index i od 0 do 255 sa najprv načíta 1 bajt **bitsCountByte**, ktorý udáva koľkými bitmi je frekvencia tohto znaku zakódovaná. Ak je **bitsCountByte** == 0, frekvencia je 0. V opačnom prípade sa načíta toľko bajtov, aby obsahovali aspoň **bitsCountByte** bitov. Tieto bajty sa prevedú na reťazec 0/1 dĺžky 8***codeBytes**, z ktorého sa prvých **bitsCountByte** bitov postupne interpretuje ako binárny zápis celého čísla - výsledná hodnota je frekvencia príslušného znaku.

Po zrekonštruovaní frekvenčnej tabuľky sa zavolá **buildTree()**, ktorá vytvorí rovnaký Huffmanov strom, aký bol použitý pri kompresii. Zvyšok súboru tvorí bitový stream zakódovaného textu - ten sa načíta po bajtoch, pričom každý bajt sa konvertuje na 8-bitový reťazec núl a jednotiek a pridá sa do reťazca bits. Pri dekódovaní algoritmus vždy začne v koreni stromu a pre každý bit vykoná prechod doľava (pri 0) alebo doprava (pri 1). Keď dojde na list, zodpovedajúci znak sa zapíše do výstupného súboru a pre ďalší bit sa opäť začne od koreňa. Počet bitov spracovaných v tomto cykle je obmedzený na **textBits**, takže sa ignorujú paddingové nuly na konci. Takto sa obnoví pôvodný text, ktorý bol komprimovaný Huffmanovým kódovaním.

Zložitosti (T a O – uvádzam ju vždy pod časovou zložitosťou):

- f = frekvencia
- m = dĺžka vstupu = počet zakódovaných bitov na vstupe

*void **decompress**(string inputFile, string outputFile)*

Vo funkcii sa nachádza 5 for-cyklov a 1 while-cyklus.

1. for-cyklus inicializuje frekvenčnú mapu pre všetky ASCII znaky, teda **T(m)** $=\sum_{i=0}^{255} 1 = 256$

O(1)

2. for-cyklus spolu s ďalšími 2 extrahuje frekvencie z hlavičky komprimovaného súboru, časová zložitosť tohoto cyklu: **T(m)**$=\sum_{i=0}^{255} 1 = 256$

a. For-cyklus zistí, koľko bitov má prečítať ďalší cyklus, aby som získala frekvenciu pre každý ASCII znak, **T(m)** = $\sum_{i=0}^{\log f} 1 = \log f + 1$

b. For-cyklus, ktorý prečíta log f bitov - tým získam frekvenciu, **T(m)**=$\sum_{i=0}^{\log f} 1 = \log f + 1$

O(log f)

3. ***BuildTree()***, **T(m)** = 511 ∈ **O(1)**

4. while-cyklus prečíta zvyšné byty a uloží ich do pamäte ako string, **T(m)** = $\sum_{i=0}^{m-1} 1 = m$, kde m je počet zakódovaných bitov

O(m)

5. for-cyklus lineárne prechádza bitmi a huffmanovým stromom - akonáhle prídem na list, tak som dekódovala znak, ten zapíšem do súboru, **T(m)** = $\sum_{i=0}^{m-1} 1 = m$

O(m)

Celková **T(m)** = 256 + 256 (2 log f + 2) + 511 + m + m = 2m + 1279 + 512 log f, kde f je frekvencia znaku, m je počet zakódovaných bitov.

Asymptotickú hornú hranicu zložitosti O dokážem z T(m):

Pre krok m:

$$2m + 1279 + 512 \log f \leq 1793m \in O(m) \; s \; c = 1793 \; a \; m_0 = 1, f \leq m, f \geq 0$$

Pre krok m+1:

$$2(m+1) + 1279 + 512 \log f \leq 1793(m+1)$$

$$2m + 2 + 1279 + 512 \log f \leq 1793m + 1793$$

$$2m + 1281 + 512 \log f \leq 1793m + 1793 \; /-2m$$

$$1281 + 512 \log f \leq 1791m + 1793 \; /-1281$$

$$512 \log f \leq 1791m + 512$$

Počet zakódovaných bitov m bude vždy väčší ako logaritmus z frekvencie znaku.

Takže asymptotická horná hranica zložitosti celého algoritmu je $\boldsymbol{O(m)}$.

Asymptoticky neviem program zrýchliť, no napríklad operácie so stringami pri dekódovaní sú časovo aj pamäťovo náročné, namiesto toho je lepšie použiť bit buffer a bitové posuny - tým by som mohla zlepšiť program.